Scoping an FDE Project, End to End: A Beginner's Guide

Oct 3, 2026 · @Sourav Banerjee

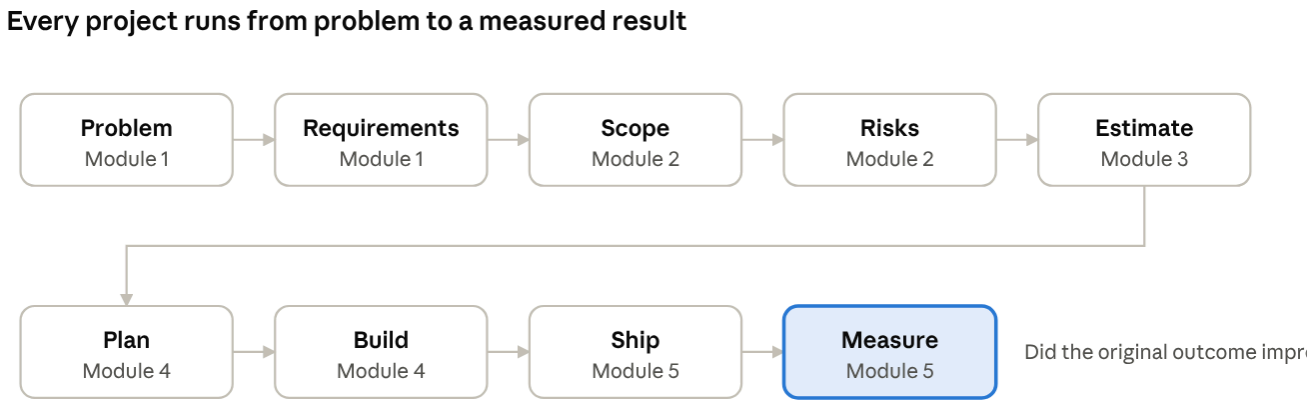
Scoping means working out what you are really building, how hard it is, and how long it will take, before you promise anything. This guide walks through the whole journey in plain language, in the order a real project unfolds.

How to use this guide

- **Read it top to bottom.** Each module builds on the one before it.
- **Every topic is short.** You get the idea, a simple example, and a "what to remember" line.
- **No jargon needed.** Where a technical word shows up, it is explained in everyday terms.
- **Two case studies and a checklist at the end.** They let you see the whole method working on real-sounding requests.

The journey at a glance

- **Module 1: Understand the request.** Find the real problem, a measurable goal, and what "done" means.
- **Module 2: Figure out what you are building.** Screens, data, states, workflows, complexity, and hidden work.
- **Module 3: Estimate honestly and shrink.** Size AI work properly, cut to a first release, and give a trustworthy range.
- **Module 4: Turn the estimate into a plan.** Milestones, critical path, ownership, and scope changes.
- **Module 5: Roll out and prove the result.** First users, monitoring, learning from failures, and closing with evidence.
- **Module 6: See the method in action.** Two case studies and a 30-minute practice exercise.
- **Module 7: Run the project day to day.** Documents, questions, meetings, clear speaking, and warning signs.
- **Module 8: The pre-estimate checklist.** Everything to check before you give anyone a date.

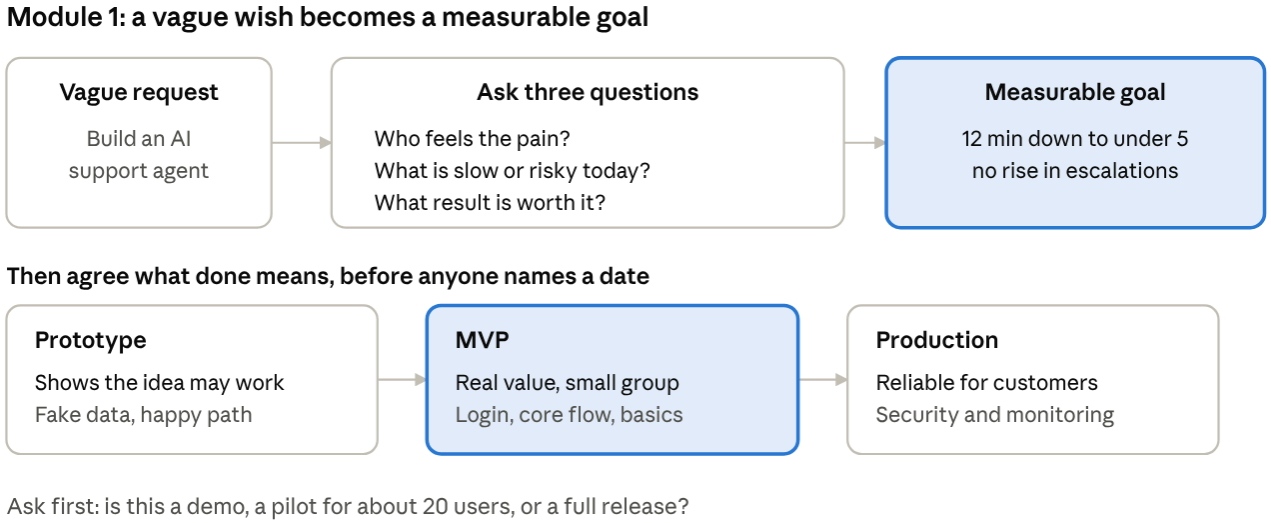


project journey, 9 steps

Read it left to right, then continue on the second row. Click any label to edit it or leave a comment.

Module 1: Understand the Request Before You Build Anything

Most projects go wrong in the first week, not the last one. This module is about getting clear on what is being asked before anyone writes code.



Module 1 infographic

Topic 1.1: Draw the map first

The idea in simple words

- A developer asks, "How do I build this feature?"
- A project lead asks a bigger question: "What should we build, how long will it take, what could go wrong, and how will we know it worked?"

- Your job as the lead is to create that clarity so the team does not have to guess.

What goes wrong without a map

- The team starts coding the happy path right away.
- Halfway through, someone finds a rule nobody mentioned.
- Later, a connection to another system turns out to be harder than expected.
- The date slips, and everyone argues about whether the project is "finished".
- Nobody was careless. They just never agreed on what the work actually was.

What a good map shows

- What is included right now.
- The assumptions behind your estimate.
- The questions that are still open.
- The signals that would make you change direction.

The map will never be perfect, and that is fine. Its job is to give everyone the same picture and to be updated as you learn more.

What to remember: Senior people do not predict the future. They say their assumptions out loud, spot the unknowns early, and update the plan when new facts show up.

Topic 1.2: Build a delivery model

The idea in simple words

A delivery model is just a chain that connects the business goal to the actual work. It runs like this: Problem, then Requirements, then Scope, then Risks, then Estimate, then Plan, then Build, then Ship, then Measure.

Start with the real problem, not the solution

A customer might say, "We want an AI support agent." Do not start designing from that sentence. First find out three things:

- **Who** feels the pain: the customer, the support agent, the operator, or the manager?
- **What** is slow, costly, risky, or frustrating today?
- **What result** would make the project worth doing?

Make the goal measurable

- Too vague: "Build an AI assistant for support."
- Clear enough to start: "Bring average ticket handling time from 12 minutes down to under 5, without more escalations or complaints."

Once the goal is measurable, your technical questions suddenly have a purpose. Which ticket types count? Can the system only read data, or can it change accounts? How many mistakes are acceptable? Where must a human approve things?

Agree on what "done" means

A demo, a small pilot, and a real production service are three different projects. Settle which one you are building before anyone mentions a date.

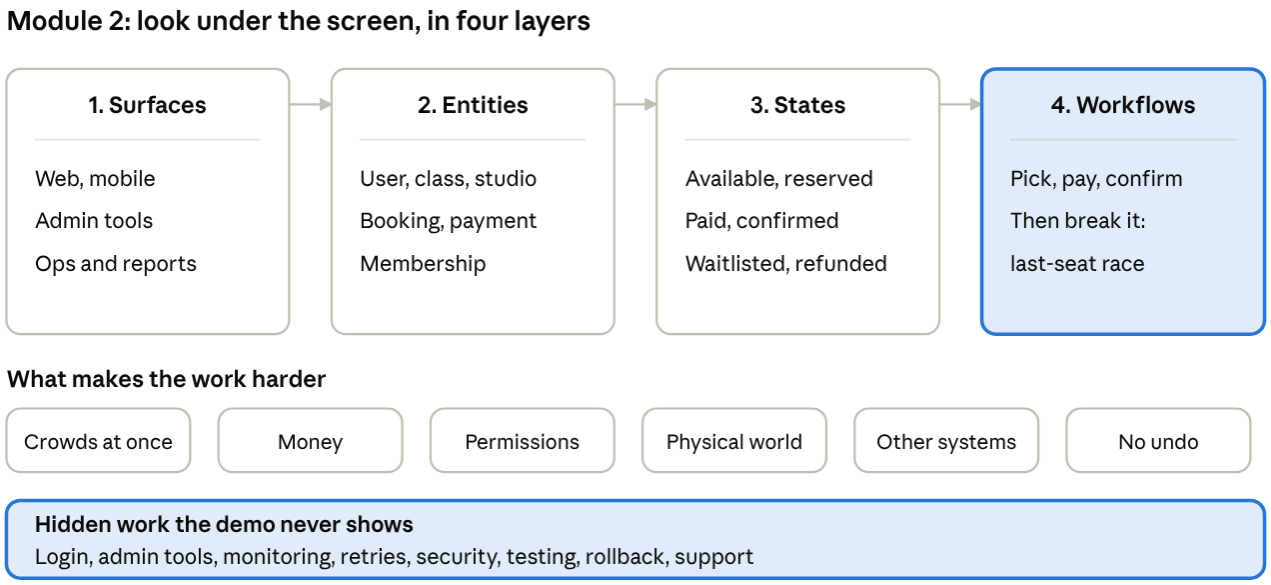
Level	What it is for	What it usually includes
Prototype	Show the idea might work	Manual setup, small sample of data, only the smooth path
MVP (first real version)	Give real value to a small group of real users	Login, the core workflow, basic error handling, simple operations
Production	Run reliably for real customers	Security, monitoring, support, rollback, scale, recovery, compliance

The one question to ask before talking dates: "When you say done, do you mean a demo, a pilot for about 20 users, or a full release?"

What to remember: Delivery starts with a measurable outcome and a shared meaning of "done".

Module 2: Figure Out What You Are Really Building

Number of pages is a poor way to measure effort. This module teaches you to look underneath the screens and see the real system.



Module 2 infographic

Topic 2.1: Scope what you are actually building

The idea in simple words

Start with what the user sees, then keep digging downward. Work through four layers in this order: Surfaces, then Entities, then States, then Workflows.

Step 1: List the surfaces (what people touch)

- Web: landing pages, dashboards, customer flows.
- Mobile: iOS, Android, notifications, permissions.
- Admin: settings, support tools, content, refunds.
- Operations: reports, alerts, reconciliation, moderation.

Step 2: List the entities (the "nouns" of the business)

For a class-booking product these might be user, instructor, class, class session, studio, room, membership, booking, and payment.

Step 3: Write down the states (how things change)

- A booking is not just "booked". It can be available, reserved, paid, confirmed, attended, waitlisted, promoted from the waitlist, or refunded.
- Every change between states comes with rules: who can trigger it, what must be true first, what gets recorded, and what happens if it fails.
- So one simple "Book" button can hide a whole set of rules behind it.

Step 4: Trace the workflows

Walk through each important journey in plain words before you draw any architecture. For example: the user picks a class, the system checks space, checks membership, takes payment, holds the seat, confirms the booking, sends a message, and updates the admin dashboard.

Then deliberately break it:

- Two people try to grab the last seat at the same moment.
- The payment goes through but saving the booking fails.
- The instructor cancels an hour before class.

These are not rare oddities. They are part of the product.

What to remember: Surface, then Entities, then States, then Workflows. This shows you the system hiding behind the screen.

Topic 2.2: Read the complexity before you estimate

The idea in simple words

A project is hard when many rules, systems, and failure paths interact, not when it simply has many pages. A big content website can be routine. A small booking flow can be tough.

A rough way to think about it

Complexity grows with the number of entities, state changes, outside systems, ways things can fail, and real-world production demands. Treat this as a checklist, not as maths.

Example: content site vs booking site

Question	Content platform	Booking platform
What changes?	Viewed, completed, subscribed	Available, reserved, paid, cancelled, refunded, waitlisted
Does it need to be real-time?	Rarely	Yes, when seats are limited
What does it connect to?	Login, billing, content	Payments, calendar, notifications, door access, maps
What does a failure cost?	Progress may not save	Double charge, overbooking, a locked door
What does running it involve?	Managing content	Scheduling, refunds, waitlists, no-shows, support

Five things that quietly push an estimate up

- **Many people at once:** lots of users changing the same thing at the same time.
- **Money:** mistakes need refunds, reconciliation, and an audit trail.
- **Permissions:** different roles can see or change different things.
- **Real-world state:** the software must match a real room, shipment, device, or person's schedule.
- **Outside systems:** you depend on tools and APIs you do not control.
- **Things you cannot undo:** some mistakes cannot be quietly fixed later.

Do not hide uncertainty behind confidence

- Weak answer: "This should take about three weeks."
- Stronger answer: "The core workflow is likely three to four weeks. The biggest risk is payment reconciliation, so I would do a two-day test run on that before committing to a final date."

A good estimate tells people where you have evidence, where you are guessing, and which unknown could move the date.

What to remember: Look for many states, systems, and failure paths colliding. That is what makes a project hard.

Topic 2.3: Account for the work the demo does not show

The idea in simple words

A demo shows the happy path. The real project includes everything that keeps the product safe and running. The hidden work often decides whether the system is trustworthy.

What I check before committing to a date

- Login and account recovery.
- Roles and permissions.
- Admin and support tools.
- Moving or loading old data.
- Analytics and success measures.
- Logging, tracking, and alerts.
- Safe retries so the same action is not done twice.
- Limits against abuse and overuse.
- Security and privacy review.
- Testing and a practice (staging) environment.
- Rollout and rollback plan.

- Support and incident response.

If support cannot run it, it is not done

Imagine customers can book a class, but support cannot cancel a booking, refund a payment, change capacity, or see why a confirmation failed. The customer screen works, yet the service is not usable in real life.

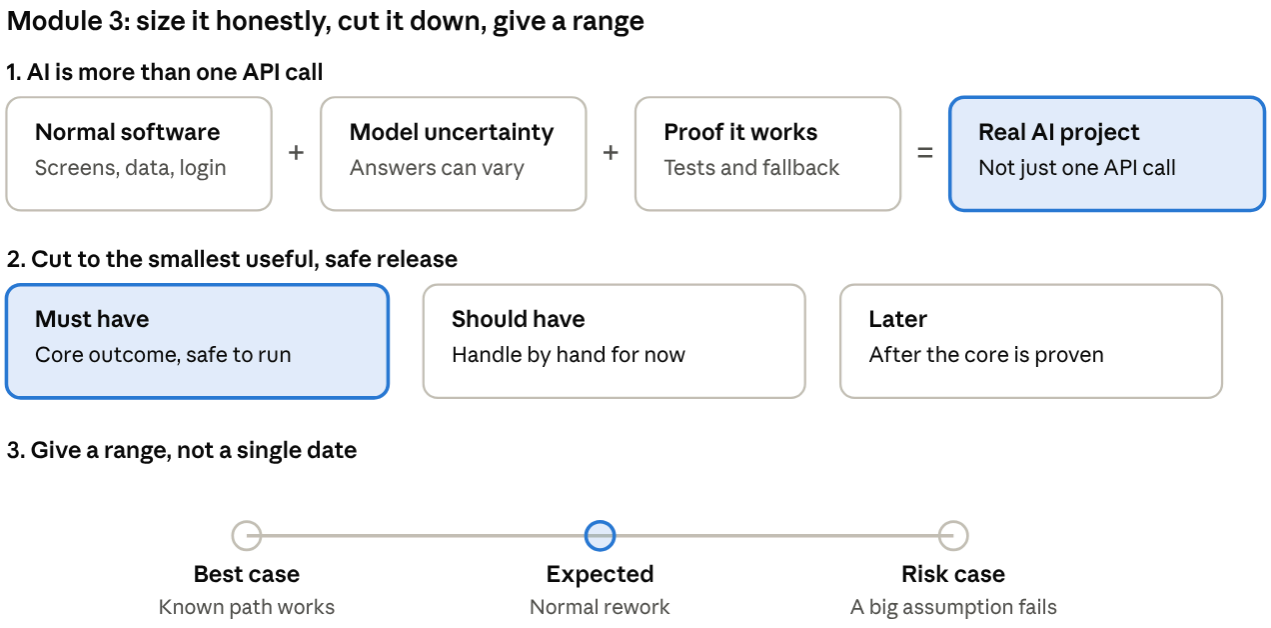
Mini exercise: find the hidden work

A manager asks for a page where people upload a resume and get AI feedback. Name at least eight tasks nobody will see on the screen. Some ideas: checking the file, scanning for viruses, reading the file, storing and deleting it, protecting personal data, login, handling AI failures, content safety, usage and cost limits, saving feedback history, analytics, support tools, monitoring, and tests.

What to remember: See the work that is invisible in the screenshot. That is where production quality lives.

Module 3: Estimate Honestly and Shrink to a First Release

Now that you know what is being built, this module covers how to size it fairly, how to trim it down, and how to turn it into a number people can trust.



Module 3 infographic

Topic 3.1: Estimate AI as a product, not a single API call

The idea in simple words

AI adds two extra jobs that normal software estimates often forget: keeping unpredictable behaviour under control, and proving the behaviour is good enough.

Why the first demo is misleading

The first call to an AI model might take an afternoon. The actual project is everything around it: getting to the right data, setting boundaries for what the AI may do, testing quality, having a human fallback, tracking what happened, controlling cost, handling failures, and improving bad answers over time.

Two kinds of logic

- **Predictable logic:** if the payment succeeds, mark the booking as paid. Same input, same result.
- **Judgement-based logic:** decide if an interview answer is strong. Several reasonable answers exist, so you need a way to measure quality.

Questions your estimate must answer

- What exactly should the AI decide or write?
- What information does it need, and where does that come from?
- What actions is it allowed to take?
- Which actions need a human to approve first?
- How many mistakes are acceptable?
- How will quality be measured before and after launch?
- What happens when the AI is unsure, slow, down, or too expensive?

Example: a mock interview assistant

Version	What is included	Rough size
Prototype	Simple chat screen, one prompt, a fixed interview flow, a final feedback message	A few days
MVP	Login, saved sessions, several modes, document lookup, scoring guide, structured report, history	Several weeks
Production	Test set, tracking, prompt versions, regression tests, cost controls, fallback, analytics	One to several months

Looking up documents is really data work

How well the AI finds the right information depends on the quality of the source material. Plan time for cleaning documents, access rules, labels, splitting into chunks, keeping things fresh, showing sources, and testing the search. If nobody owns updates, the system quietly goes out of date.

Write the tests before the pilot

Before real users try it, agree on sample cases and the expected behaviour. Include clean examples, unclear requests, missing information, unsafe requests, and cases that should be passed to a human. After launch, turn every important failure into a new test.

The loop looks like this: Build, then Evaluate, then Observe, then Collect failures, then Improve. AI quality comes from this loop, not from one perfect prompt.

What to remember: "The model can do it" is where the AI project begins, not where it ends.

Topic 3.2: Cut scope without losing the outcome

The idea in simple words

A good lead does not accept a huge wish list and just attach a big number to it. Your job is to find the smallest release that gives real value and removes the biggest risk.

What an MVP really is

An MVP should be narrow, not sloppy. It works for a clearly defined group of users, is safe to run, and answers a real question about the product or the way you are building it.

Sort every request into three buckets

- **Must have:** needed to deliver the core outcome or to run safely.
- **Should have:** useful soon, but can be handled by hand for now.
- **Later:** valuable once the core workflow has proven itself.

Example: class-booking product

In the first release	Saved for later
Responsive website	Native iOS and Android apps
Class schedule and booking	Community feed
Hosted payment checkout	Movement-feedback feature
Simple admin portal	Automatic studio door access
Email confirmation	Advanced loyalty and membership rules

Keep manual steps where they teach you something

- Automate the core value, not every task around it.
- Early on, support staff can approve refunds, an operator can upload a spreadsheet, and a small script can create the weekly report.

- Replace these steps with automation only when volume or error rates justify it.
- Manual steps still need an owner, a safe process, and a way to measure the workload.

Retire the biggest risk first

Pick the first release around the assumption most likely to kill the project. If nobody has proven that people want interview feedback, test that before building community features, complicated billing, or a big content platform.

A simple way to compare options for version 1

The value of an MVP rises with how much core value users get and how fast you learn, and falls with how much it costs to deliver.

Mini exercise: You have six weeks to build an enterprise document assistant. The request includes chat, Slack, email, permissions, analytics, many languages, document editing, and automatic actions. Decide what a six-week MVP looks like.

What to remember: Scope, then Prioritise, then Reduce, then Deliver. Owning a project includes being able to say "not yet".

Topic 3.3: Turn scope into an estimate people can trust

The idea in simple words

A good estimate is one another engineer can read, question, and challenge. It is a range with visible assumptions, not one confident number.

Step 1: Break the work into deliverables

- Product clarification and technical design.
- Frontend and user experience.
- Backend and data.
- Connections to other systems and infrastructure.
- AI workflow and testing of its quality.
- Admin and operations.
- Testing, rollout, and monitoring.

Step 2: Separate what you know from what you are guessing

Type	Example	How to handle it
Known	Build a standard dashboard with login	Estimate from experience
Known unknown	Can the payment provider support our refund rules?	Do research or a short technical test run (a spike)

Type	Example	How to handle it
Unknown unknown	A surprise limit in a vendor's system	Keep a buffer and connect to risky things early

Step 3: Give a range and say what moves it

- Early on, "21 working days" is usually made-up precision.
- "Three to five weeks" is more honest, as long as you can explain what keeps it near three and what pushes it toward five.
- Think in three cases: the best case (known path works), the expected case (normal rework and review), and the risk case (a big assumption fails or scope grows).
- Uncertainty should shrink as the team learns and connects things together.

Step 4: Write the assumptions down

Example of a usable estimate: "For one experienced engineer, the MVP is roughly three to five weeks, assuming hosted login, hosted payment checkout, and an existing design system. A production pilot is roughly six to eight weeks. The biggest unknowns are refund rules and door-access integration."

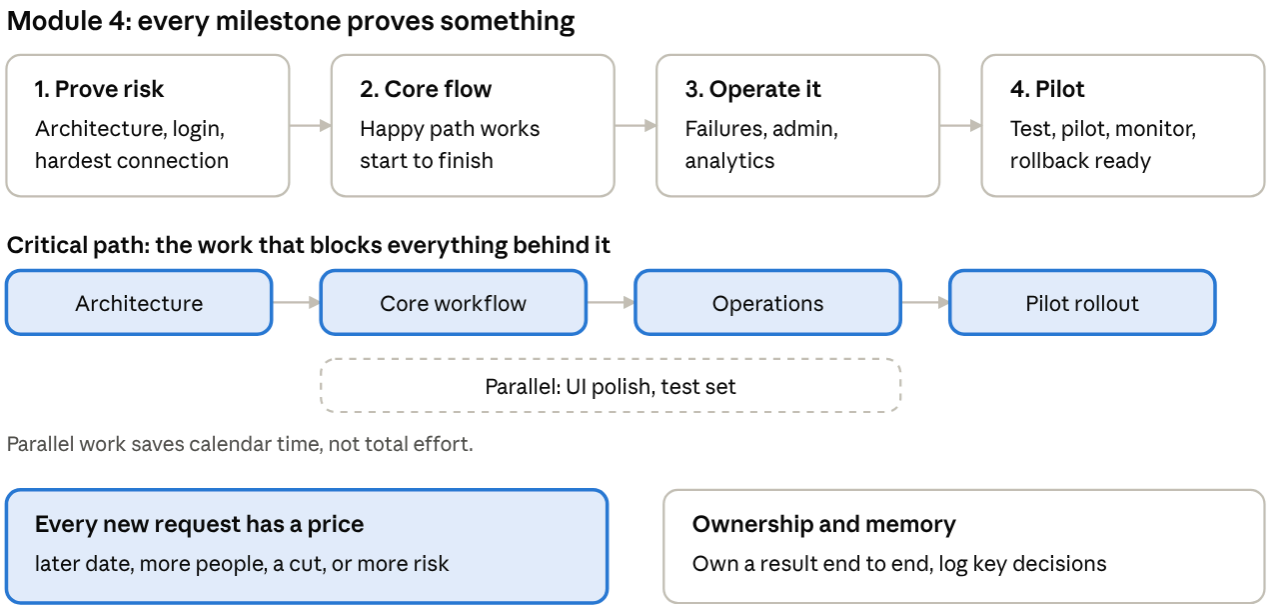
Step 5: Re-estimate when facts change

- Do not defend an old number after its assumptions have failed.
- Re-estimate after the first integration, after a technical test run, when scope changes, or when the target moves from demo to pilot.
- Explain why: "The estimate moved from four weeks to six because the new requirement adds custom permissions and a data migration. Here is the added work and the trade-off available."

What to remember: A defensible estimate is not a confident number. It is a range with visible assumptions and risks.

Module 4: Turn the Estimate Into a Build Plan

An estimate tells you how much work might exist. A plan shows how the team will cut risk step by step and reach a usable release.



Module 4 infographic

Topic 4.1: Milestones that prove something

The idea in simple words

A milestone is not "end of week two". A real milestone gives you a working slice, proves a risky assumption, or unlocks the next chunk of work.

A simple four-step milestone pattern

- **Milestone 1:** check the architecture, the data, login, and the hardest connection to another system.
- **Milestone 2:** get the core happy path working from start to finish.
- **Milestone 3:** add failure handling, admin operations, and analytics.
- **Milestone 4:** test, run a pilot, monitor, and get the rollback ready.

Topic 4.2: Protect the critical path

The idea in simple words

Some work can run side by side, and some work blocks everything behind it. The chain of blocking work is called the critical path.

- Connect the riskiest dependency early.
- Keep the critical path visible to everyone.
- Do not let optional polish push aside the work on the path.
- Example path: architecture, then core workflow, then operations, then pilot rollout.
- Doing things in parallel saves calendar time, but it does not reduce the total effort.

Topic 4.3: Give people end-to-end ownership

- Do not cut the work so finely that everyone owns a layer and nobody owns the result.
- One engineer might own booking from the user's click all the way to confirmation and support visibility.
- Another might own payment through to reconciliation and alerts.

Topic 4.4: Make scope changes visible

The idea in simple words

Change is normal. Silent change is what breaks plans. Every new requirement should arrive with its price tag.

- If scope goes up, then time goes up, the team grows, or quality and risk change.
- You cannot add scope while pretending everything else stays fixed.
- Say it plainly: "We can add that, but it needs a later date, more people, or dropping something else."

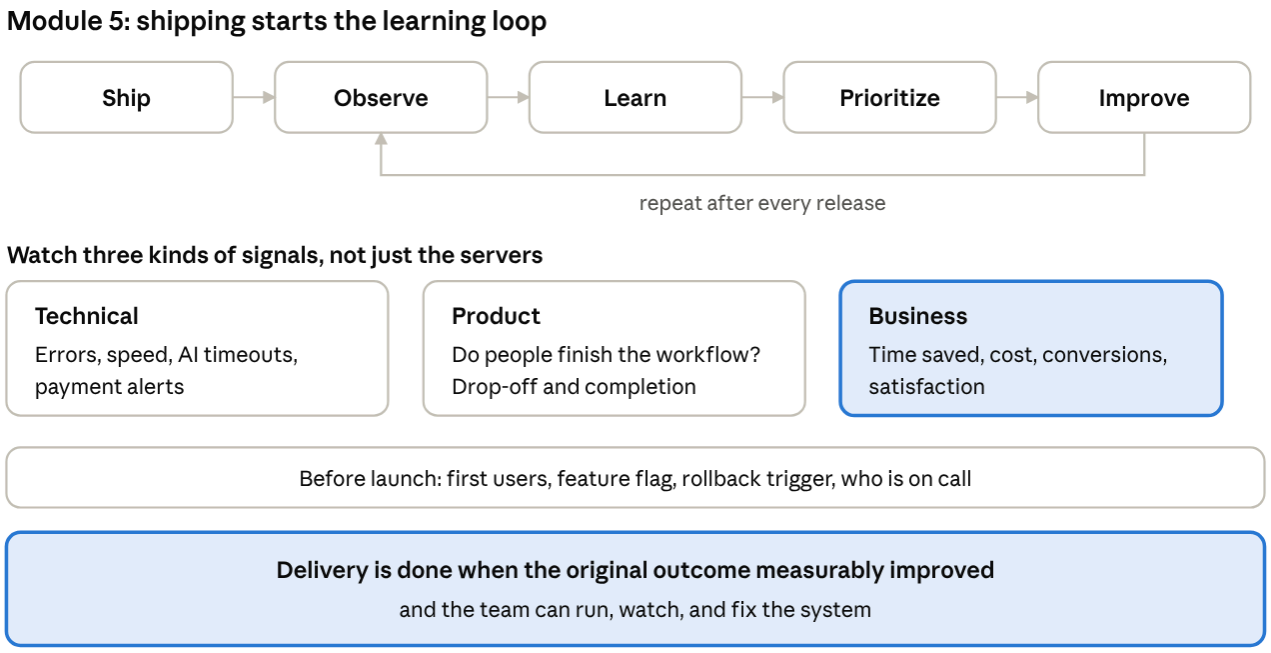
Topic 4.5: Keep a short decision log

Write down the decisions that change the shape of the project, such as hosted service vs custom build, manual vs automated, one customer vs broad rollout, data design choices, and safety limits. A few sentences with clear reasoning are enough.

What to remember: Plan milestones, dependencies, ownership, and decisions, not just a list of tickets.

Module 5: Roll Out, Operate, and Prove the Result

Shipping is not the finish line. Delivery is the result the system produces in real use.



Module 5 infographic

Topic 5.1: Choose first users and a way back

Before launch, answer these:

- Who gets the feature first?
- Can it be switched on and off with a feature flag?
- Which numbers will we watch?
- What conditions trigger a rollback?
- Who is on hand if something fails?

Topic 5.2: Watch the workflow, not just the servers

Healthy servers do not guarantee a healthy product. Track three kinds of signals:

Kind	What to watch
Technical	Error rate, speed, queue length, AI timeouts, payment notification failures
Product	Completion rate, drop-off, successful bookings, finished interview sessions
Business	Time to resolve, support cost, conversions, retention, customer satisfaction

Topic 5.3: Turn production failures into planned work

- The loop is: Ship, Observe, Learn, Prioritise, Improve.
- Real use will prove parts of your original plan wrong. That is normal.
- Before fixing a failure, sort it: was it product design, code, data, operations, search/retrieval, AI behaviour, documentation, or user training?

Topic 5.4: Report what changed, not only what shipped

A good wrap-up compares the result with the original target and names the next bottleneck. Example: "We piloted the support assistant with 25 agents. Median resolution time fell from 12 minutes to 7, and escalations stayed within 1 percentage point of normal. The biggest failure type was outdated policy lookups, so the next step is automatic knowledge refresh and tests for lookup quality."

What to remember: Delivery is done when you have evidence the original problem improved and the team can operate the system.

Module 6: See the Method in Action

Two worked examples and one practice exercise. Notice that the same steps repeat every time.

Topic 6.1: Case study, a class-booking product

The request: "Build a class-booking product with web and mobile apps, payments, memberships, community features, movement feedback, and automatic physical locations." It sounds like a few screens, but it quickly turns into scheduling, payments, operations, and real-world limits.

Step 1: Ask what business this really is

- Is the main goal finding classes, filling studios, selling subscriptions, or AI instruction?
- Is the first market one studio or many locations?
- Does the business already have instructors, schedules, and payment rules?
- Is AI coaching needed at launch or is it a later dream?

Step 2: Map the product and the system behind it

- **Customer side:** discovery, schedule, booking, payment, membership, profile.
- **Operations side:** class management, capacity, refunds, instructors, locations, support.
- **Platform side:** login, notifications, analytics, mobile release, security, monitoring.

Step 3: Find assumptions that could double the work

- How complex are the membership, cancellation, refund, and waitlist rules?
- What does "AI coaching" really mean: generic tips, body-pose detection, or live computer vision?
- How does automatic door access connect to physical devices?
- Do we need fully native mobile apps, or can one shared codebase work?

Step 4: Draw a hard line around version 1

Recommended first release: a responsive website, hosted login, class schedule, booking, hosted payment checkout, simple membership rules, email confirmation, and an admin portal for one location. Hold back native apps, community features, live coaching, and door automation until one location can win customers, take payment, run classes, and handle support reliably.

Step 5: Estimate by delivery stage

Stage	Rough size for one experienced full-stack engineer using modern tools	Main assumption
Visual prototype	1 to 2 weeks	Fake data, no real operations
Booking MVP	6 to 10 weeks	One location, hosted services, simple rules
Production pilot	3 to 6 months	Admin, support, monitoring, refunds, mobile, reliability
Full vision	6 to 12+ months and a team	Movement feedback, community, hardware, many locations

Step 6: Say it in a way the team can use

"The website itself is not the main effort. Booking, payment, membership, operations, mobile releases, and physical-location links create most of the scope. I would aim for six to ten weeks for a single-location web MVP, then re-estimate mobile, AI coaching, and automatic access after short technical test runs."

What this case teaches: Estimate the system behind the product, and keep the immediate MVP separate from the long-term vision.

Topic 6.2: Case study, an internal support copilot

The request: "We have too many support tickets. Build an agent that answers customers and completes common support tasks." Several projects hide in that sentence. Drafting an answer for an employee is one risk level, sending it straight to a customer is higher, and

changing an order or giving money back is higher still. Treat each as a separate scope decision.

Step 1: Sit with the current workflow

- Watch agents resolve real tickets. The official process is helpful, but real clicks, hand-offs, workarounds, and exceptions are what you build against.
- Ask where tickets arrive, how they are sorted, which systems agents open, which steps need judgement, which are policy-driven, which categories are high volume but low risk, and which mistakes cause money, legal, or customer harm.
- Expect the real workflow to be messier than the diagram: copy-pasting, missing data, hidden spreadsheets, and manual approvals.

Step 2: Pick one low-risk, high-volume case

Say order-status questions are about 30 percent of volume. They are repetitive, measurable, and read-only, so they make a sensible pilot. Refunds and fraud do not. A good target: draft a sourced answer for order-status tickets, cut agent handling time by 40 percent, and keep factual accuracy above 95 percent on an approved test set.

Step 3: Write the boundary in plain English

- **In scope:** order-status tickets, read-only order lookup, a draft reply, sources, human approval.
- **Out of scope:** refunds, address changes, cancellations, fraud, sending directly to customers.
- **Fallback:** low-confidence or unsupported cases stay in the existing agent queue.

Step 4: Keep live data separate from reference knowledge

Source	What it is for	How it is used	Main risk
Order system	Current order and shipping state	Live lookup	Wrong customer or stale answer
Support policy	Approved explanations and escalation rules	Search over documents	Outdated or irrelevant passage
Customer message	What they want	Input to the AI	Unclear wording or trick instructions
Ticket history	Earlier contacts and promises	Background context	Too much personal data exposed

Step 5: Keep the escape hatch

The flow: receive ticket, work out the intent, check eligibility, fetch the order and policy, write a draft with sources, human review, send, log the result. If the case is unsupported or the confidence is low, or the reviewer rejects the draft, it goes to the old manual queue. A safe pilot keeps humans involved and keeps the old path alive.

Step 6: Build the test set before connecting anything

Collect realistic tickets: normal ones, missing data, policy exceptions, angry wording, unsafe requests, and ones that must go to a human. Protect customer data. Then judge on six things: did it find the right policy, is the draft factually right, did it answer, ask, or escalate correctly, did it follow the rules, is it clear and useful, and did it actually save time without adding review work.

Step 7: Estimate the pilot, not the dream

Workstream	Rough effort	Key assumption
Understanding the workflow and getting data access	1 week	Customer provides experts and test access quickly
Core agent and order lookup	1 to 2 weeks	Order API is documented and stable
Policy documents search	1 week	Policies are in one place and current
Testing and improving	1 to 2 weeks	Past examples are available
Support-platform connection	1 to 2 weeks	The platform supports draft workflows
Pilot rollout and monitoring	1 week	Small pilot group, human review stays

Overall: roughly five to eight weeks for a small team. Biggest risks are access to customer data, the support-platform link, and policy quality. Fully automatic actions are not included and would need a separate safety phase.

Step 8: Earn autonomy in stages

- **Offline:** run on past tickets without touching real users.
- **Shadow:** write drafts while agents keep using the normal process.
- **Assisted:** show drafts to a small group who review and edit.
- **Expanded:** grow traffic only when quality and operations look stable.
- **Autonomous:** consider only narrow, reversible actions with strong controls.

What this case teaches: Do not start by making the agent powerful. Start by making the workflow clear, measurable, narrow, and safe.

Topic 6.3: Practice, build a delivery model in 30 minutes

Scenario: A property-management company wants an AI assistant that answers tenant questions, schedules maintenance, and flags urgent requests. Work on paper in six short parts.

1. **Clarify the project (5 minutes).** Who are the users? What is the most painful workflow today? What measurable outcome matters? Which request types are high volume and low risk? Which actions could cause harm if wrong?
2. **Draw the system (10 minutes).** Make four lists: entities (tenant, property, unit, request, vendor, appointment, lease), states (new, triaged, scheduled, assigned, completed, escalated), connections (property system, calendar, text, email, vendor system), and roles (tenant, coordinator, technician, manager, admin).
3. **Find failure points (5 minutes).** At least one each for data (wrong tenant), scheduling (double-booked technician), safety (a gas leak treated as routine), communication (wrong appointment time), and AI (invented lease rule).
4. **Define the MVP (5 minutes).** Answer approved policy questions, sort maintenance requests, spot emergency keywords, and create a draft work order for staff approval. Do not auto-dispatch vendors, handle complex lease disputes, or close emergencies without a human.
5. **Estimate (10 minutes).** Make a table with workstream, range, assumption, and risk for discovery, knowledge assistant, request sorting, work-order connection, and pilot.
6. **Give the lead answer.** Restate the outcome, name the MVP, give the range, state assumptions, name top risks, explain the first technical test, and describe the rollout and success measure.

Sample lead answer, in short: Start with a staff-assisted pilot for one property rather than a fully automatic agent, for roughly six to nine weeks, assuming usable APIs and central policies. In week one, test the property API and build an emergency test set. Success means routine handling time drops while emergency detection stays above the agreed level.

Keep practising: Take apart two products each week. Save your first scope and estimate, then revisit it after you learn more. Judgement grows through repetition, not by memorising one design.

Module 7: Run the Project Day to Day

You do not need a heavy process. You need a few lasting documents, clear decisions, and a rhythm that keeps risk in view.

Topic 7.1: Four light documents are usually enough

1. **One-page project brief:** problem, user, outcome, scope, what is not included, major risks, owner, and target milestone.
2. **Technical design:** how it is built, how data flows, the connections, the states, trade-offs, security, rollout, and open questions.
3. **Execution plan:** milestones, dependencies, critical path, owners, checkpoints, and the estimate range.
4. **Decision log:** key decisions, the alternatives, the reasoning, the date, the owner, and what would make you reconsider.

Topic 7.2: A one-page brief people will actually read

Fill in these lines: project and owner; the problem (who feels it and what happens today); the target outcome (what measurable result should change); in scope; out of scope; definition of done (prototype, MVP, pilot, or production); major assumptions; biggest risks and unknowns; the estimate range with team shape and dependencies; and the rollout (who gets it first, how you watch it, how you roll back).

Topic 7.3: Questions that expose the real work

- **Business:** Why now? Who owns the outcome? What happens if we do nothing? Which metric matters?
- **Workflow:** Show me the current process. Where are the hand-offs? What are the usual exceptions?
- **Technical:** Which systems hold the data? What actions are needed? What are the access limits?
- **Delivery:** Who must approve? Which deadline is real? What can stay manual? What can be piloted?

Topic 7.4: How to speak clearly when you are not certain

Do not hide behind "it depends", and do not fake certainty. Give the current range, the assumption behind it, and the next piece of evidence.

Situation	What you can say
Early estimate	"This is a rough range based on today's scope. I expect it to narrow after the integration test."
Missing requirement	"The timeline depends on whether the agent only reads data or can act on accounts. Those carry different risk."
Scope increase	"We can include that, but it adds permissions and support work. We should move the date or drop something else."
Technical risk	"I do not want to bury this in a buffer. The vendor API is the main unknown, so let's test it first."
Estimate change	"The estimate changed because an assumption changed. Here is the new work and the trade-off."

Topic 7.5: Use meetings to make decisions

- Before the meeting, write down the decision that is needed.
- End with the decision, an owner, and the next checkpoint.
- If nothing needs deciding or coordinating, use a document or message instead.
- Example outcome: "Decision: version 1 is read-only with human approval. Owner: Priya. Open question: can ticket history be reached through the existing API? Checkpoint: Thursday after the security review."

Topic 7.6: Own the result without doing every task

Ownership is not controlling every file. Agree on interfaces early, give engineers meaningful slices, review the riskiest work closely, and decide quickly enough that people are never stuck waiting.

Topic 7.7: Warning signs the project is drifting

- The team is coding, but nobody can state the measurable outcome.
- Every stakeholder has a different idea of the MVP.
- The estimate lists no assumptions or exclusions.
- The hardest connection is planned for the final week.
- Nobody owns admin tools or operational support.
- The AI demo looks good, but there is no test set.

- New scope sneaks in through chat messages without changing the plan.
- The team celebrates deployment but cannot say whether users succeeded.

What to remember: Good leadership is the repeated act of creating clarity about outcome, scope, decisions, risk, ownership, and evidence.

Module 8: The Pre-Estimate Checklist

Run through this list before you put a date in front of a team or customer.

- [ ] **Problem:** What problem are we solving?
- [ ] **Users:** Who experiences it?
- [ ] **Success:** What measurable outcome defines it?
- [ ] **Definition of done:** Prototype, MVP, pilot, or production?
- [ ] **Surfaces:** Web, mobile, admin, operations, APIs?
- [ ] **Entities:** What important objects exist?
- [ ] **States:** How can those objects change?
- [ ] **Workflows:** What are the main end-to-end paths?
- [ ] **Integrations:** Which outside systems are involved?
- [ ] **Failure modes:** What can go wrong?
- [ ] **AI uncertainty:** What behaviour is judgement-based rather than predictable?
- [ ] **Production needs:** Scale, speed, cost, security, reliability?
- [ ] **Hidden work:** Login, admin, migration, testing, monitoring, support?
- [ ] **Unknowns:** What needs research or a technical test run?
- [ ] **MVP:** What is the smallest useful and safe version?
- [ ] **Estimate:** What is the range, and which assumptions support it?
- [ ] **Plan:** Milestones, dependencies, critical path, ownership?
- [ ] **Rollout:** Pilot, feature flag, alerts, rollback?
- [ ] **Measure:** Did the project improve the original outcome?

The customer usually knows the outcome they want. The forward deployed engineer's job is to turn that into a route that can actually be built.

Source

Based on the Scope a Project course in the FDE Interview Handbook, summarised in my own words.